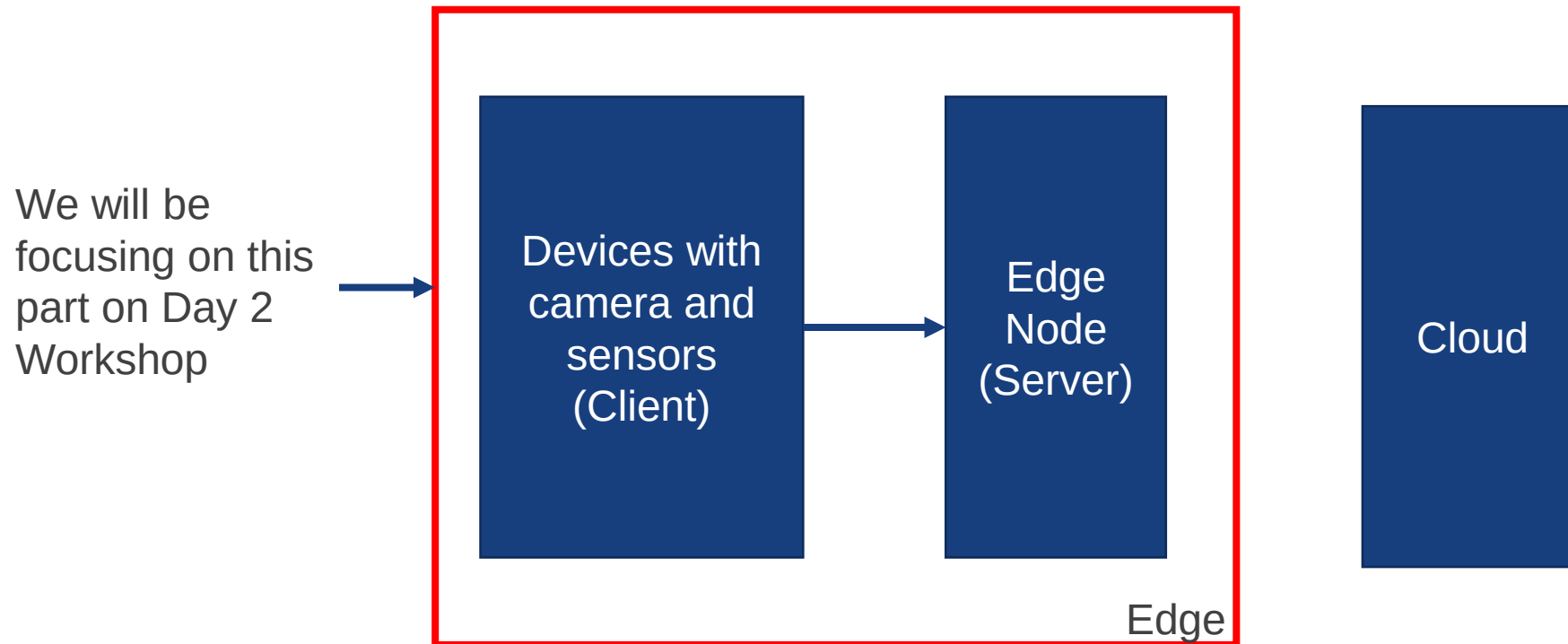


WORKSHOP DAY 2

Workshop Themes

- Day 2:
 - Learn how to process data in edge with camera
 - Learn how to send the processed data from device to edge node



Workshop Themes

- Read the images or videos from camera
- Resize the images
- Retrieve the required channel
- Filtering
- Region of Interest (ROI) detection
- Classification
- Send processed image to edge server

1.2. Reading Images from Camera

```
import cv2

# To capture video.
# Device index is set to 0 as we only have one camera
cap = cv2.VideoCapture(0)

while(True):
    # If frame is read correctly, ret will be True, vice versa
    ret, frame = cap.read()

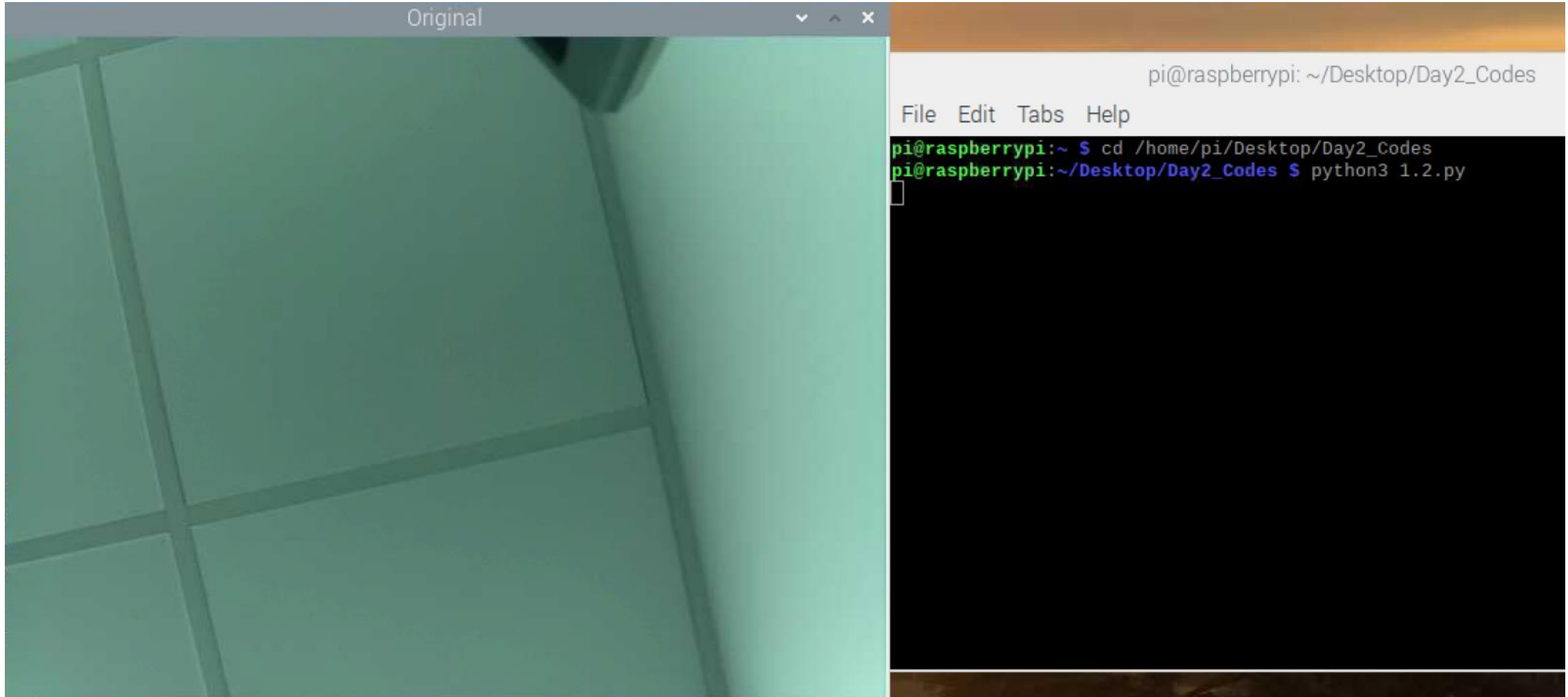
    # To display the video in new window and named it as "Original"
    cv2.imshow('Original', frame)

    # Press 'q' to quit the program
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

- Did you notice that, in vision system, **the captured video is processed frame by frame?**
- Can you imagine **if all the frames are required to send to cloud services for processing, the bandwidth cost will be tremendous and latency will be high?**

1.2. Reading Images from Camera



1.3. Resizing the Image

```
import cv2

# To capture video.
# Device index is set to 0 as we only have one camera
cap = cv2.VideoCapture(0)

while(True):
    # If frame is read correctly, ret will be True, vice versa
    ret, frame = cap.read()

    # To display the video in new window and named it as "Original"
    cv2.imshow('Original', frame)

    # To check the image information (height, width, color channels)
    print(frame.shape)

    # To resize the image (Original image, target size, interpolation
    method)
```

```
# Interpolation is a type of estimation to construct new data points
within the range of a discrete set of known data points. Common used
methods can be found from
https://docs.opencv.org/4.4.0/da/d54/group\_imgproc\_transform.html#ga5bb5a1fea74ea38e1a5445ca803ff121

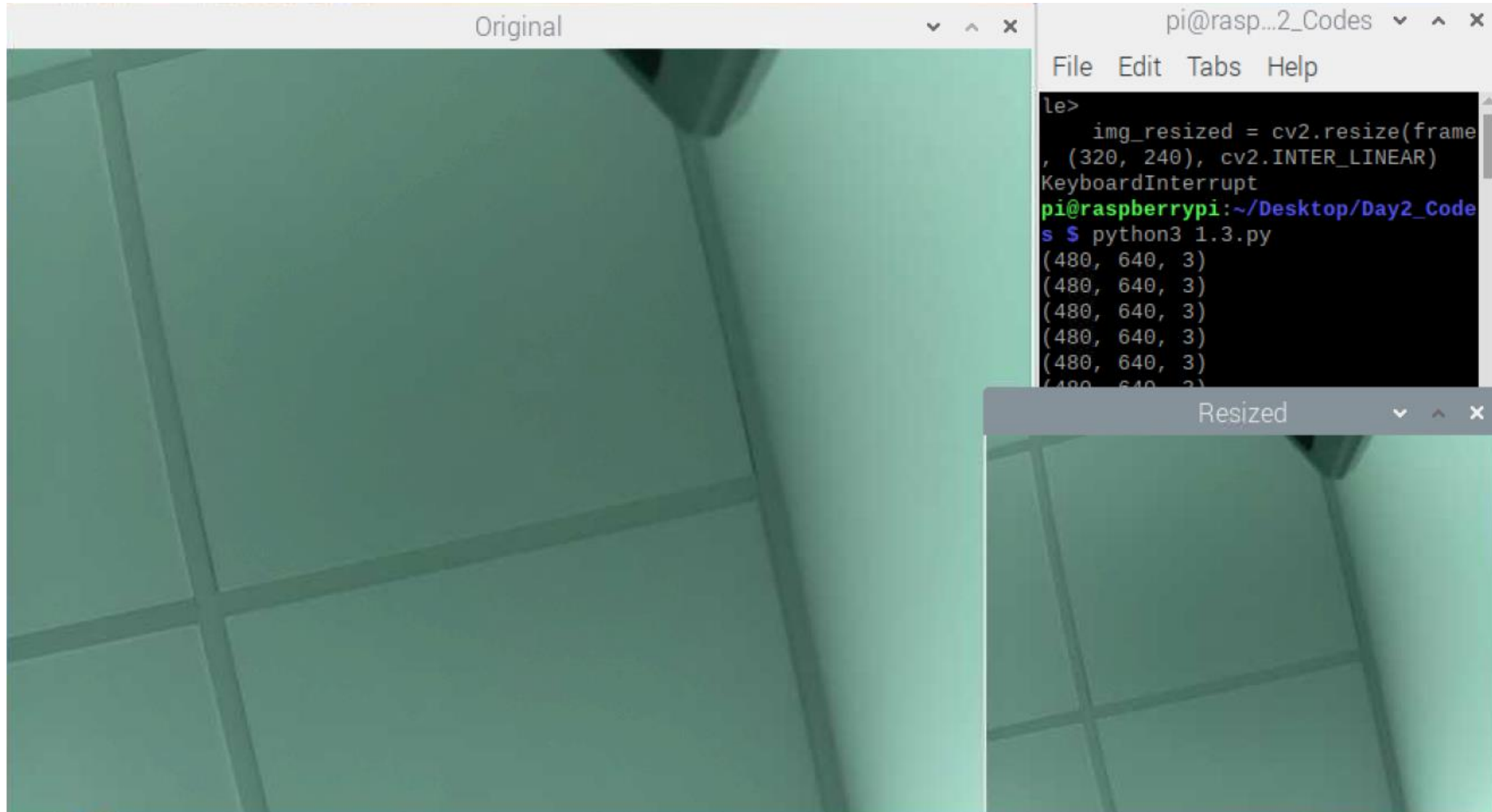
img_resized = cv2.resize(frame, (320, 240), cv2.INTER_LINEAR)

# To display the resized image
cv2.imshow('Resized', img_resized)

# Press 'q' to quit the program
if cv2.waitKey(1) & 0xff == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

1.3. Resizing the Image



1.4. Retrieving the Required Channel

```
import cv2

# To capture video.
# Device index is set to 0 as we only have one camera
cap = cv2.VideoCapture(0)

while(True):
    # If frame is read correctly, ret will be True, vice versa
    ret, frame = cap.read()
```

```
# To display the video in new window and named it as "Original"
cv2.imshow('Original', frame)

# To check the image information (height, width, color channels)
print(frame.shape)

# To convert the colour image to gray scale image
# You can find out more conversion code from
https://docs.opencv.org/master/d8/d01/group\_\_imgproc\_\_color\_\_conversions.html

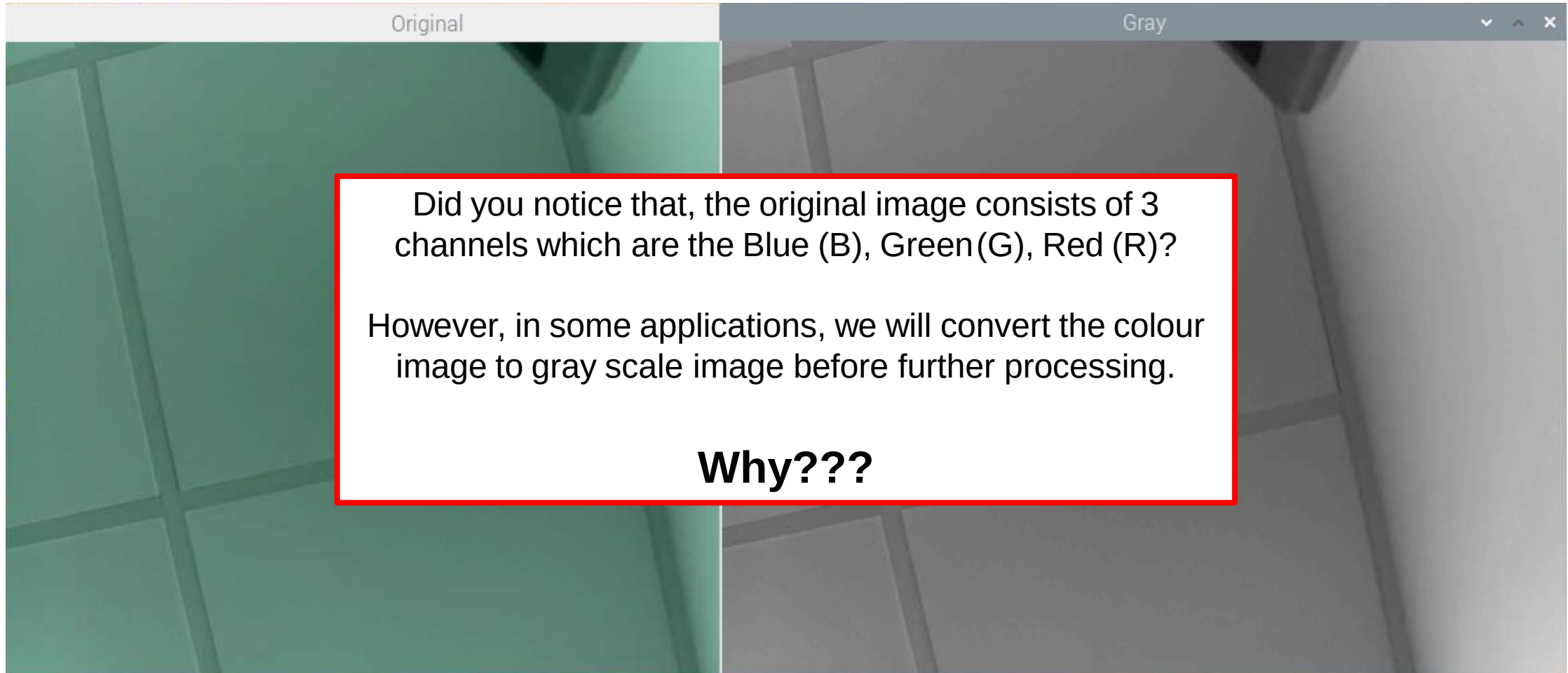
frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# To display the gray scale image in new window and named it as "Gray"
cv2.imshow('Gray', frame_gray)

# Press 'q' to quit the program
if cv2.waitKey(1) & 0xff == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

1.4. Retrieving the Required Channel



1.5. Filtering and Edge Detection

```
import cv2

# To capture video.
# Device index is set to 0 as we only have one camera
cap = cv2.VideoCapture(0)

while(True):
    # If frame is read correctly, ret will be True, vice versa
    ret, frame = cap.read()

    # To display the video in new window and named it as "Original"
    cv2.imshow('Original', frame)
```

```
# with Mean Filtering (Uncomment the code below to try)
#frame_mean = cv2.blur(frame, (11,11))
#cv2.imshow('Mean', frame_mean)
```

```
# with Gaussian Filtering (Uncomment the code below to try)
#frame_gaussian = cv2.GaussianBlur(frame, (11,11), 0)
#cv2.imshow('Gaussian', frame_gaussian)
```

```
# with Median Filtering (Uncomment the code below to try)
#frame_gaussian = cv2.medianBlur(frame,11)
#cv2.imshow('Median', frame_gaussian)
```

```
# with Canny Edges (frame, minVal, maxVal, Sobel Kernel size)
# Sobel filtering works by calculating the gradient of image intensity
at each pixel within the image
#frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
#frame_canny= cv2.Canny(frame_gray, 10, 30, apertureSize=3)

#cv2.imshow('Canny', frame_canny)
```

```
# Press 'q' to quit the program
if cv2.waitKey(1) & 0xff == ord('q'):
    break
```

```
cap.release()
cv2.destroyAllWindows()
```

1.5. Filtering and Edge Detection

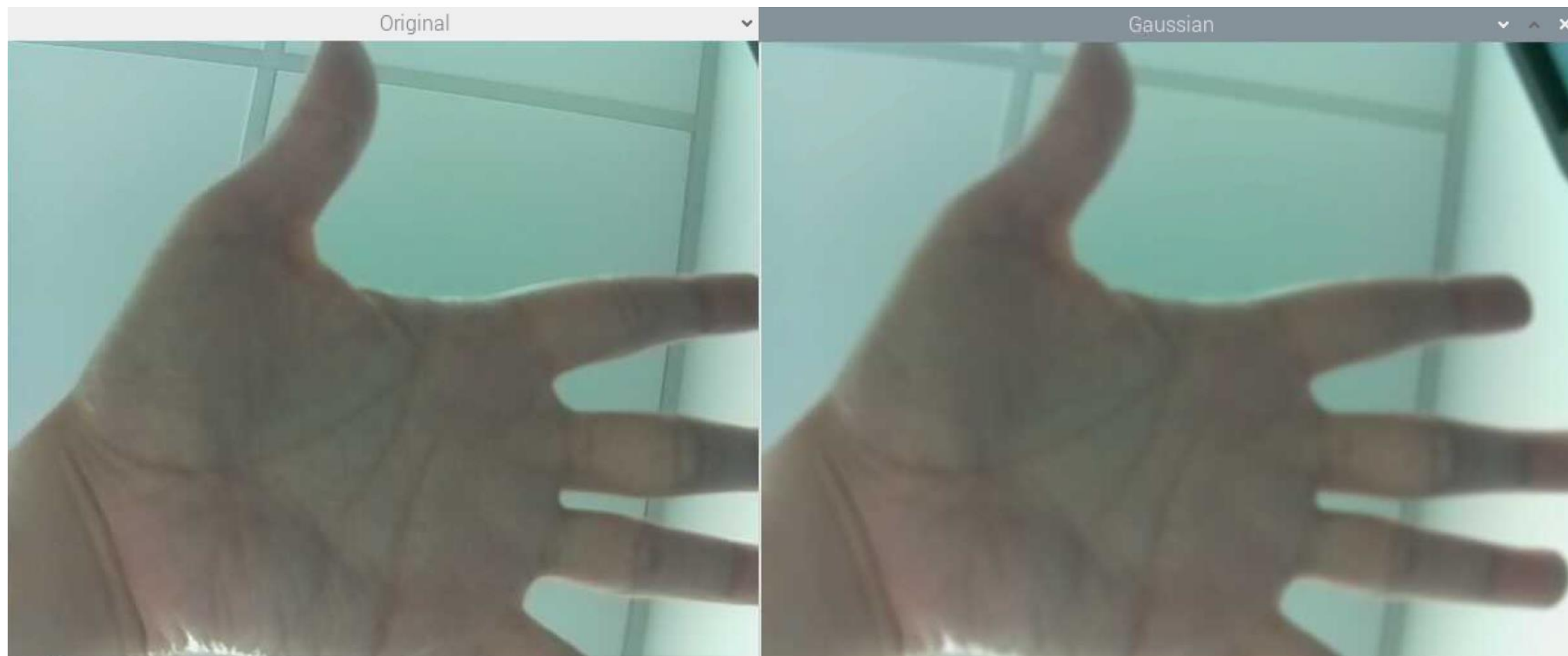
In image processing, **filtering is used to strengthen preferred features or weaken unwanted features**; Filtering can also smoothen or sharpen an image as well as to enhance edges in image

**Mean
Filtering
Example:**



1.5. Filtering and Edge Detection

Gaussian Filtering Example:



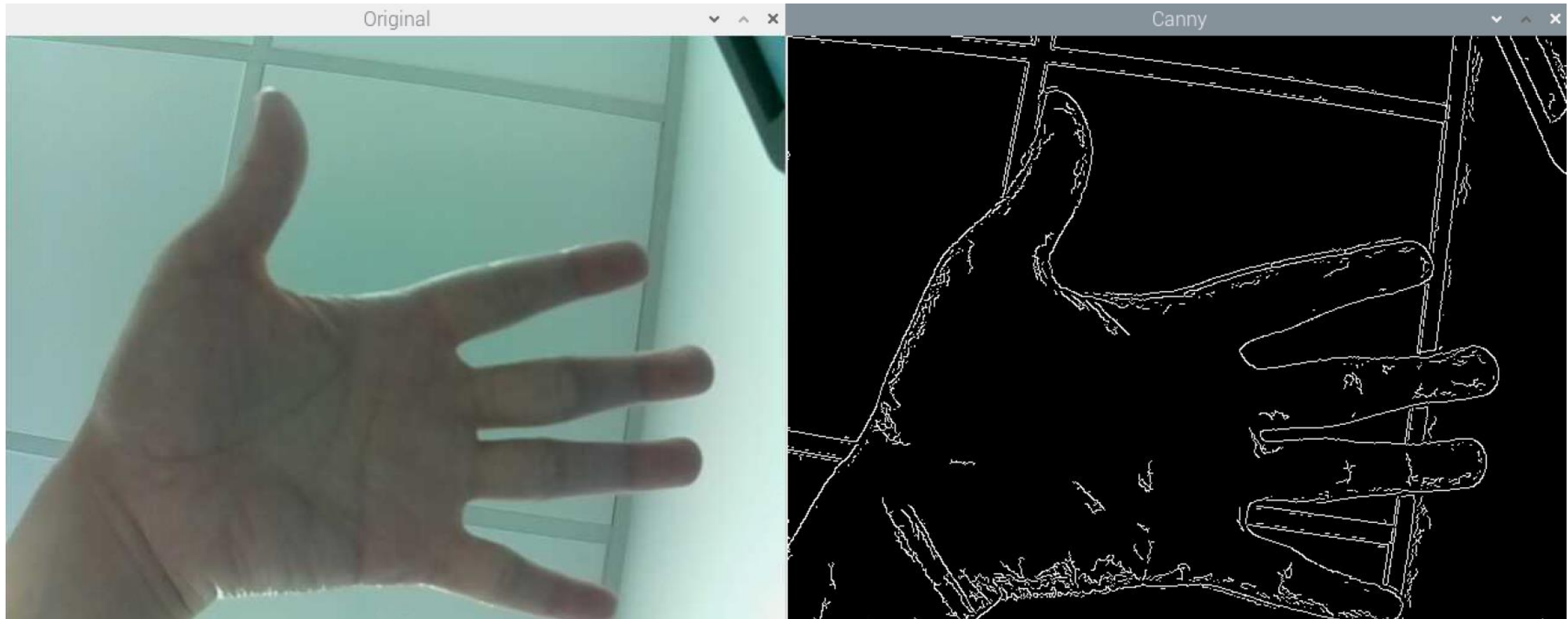
1.5. Filtering and Edge Detection

Median Filtering Example:



1.5. Filtering and Edge Detection

For edge detection, **Canny Edges** is used to find the **boundaries of objects in image**; Sobel Filtering Example:



1.6. Region of Interest (ROI) Detection

```
import cv2

# To capture video.
# Device index is set to 0 as we only have one camera
cap = cv2.VideoCapture(0)

while(True):
    # If frame is read correctly, ret will be True, vice versa
    ret, frame = cap.read()

    # Frame should convert into gray scale before perform detection
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Load the CascadeClassifier xml file
    face = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

    # Perform detection
    face_dect = face.detectMultiScale(frame_gray, scaleFactor=1.3,
minNeighbors=2)
```

- **Cascade Classifier is machine learning based approach to detect objects in images**
- Extremely fast computation, good for real-time detection and can be applied to other type of object detections

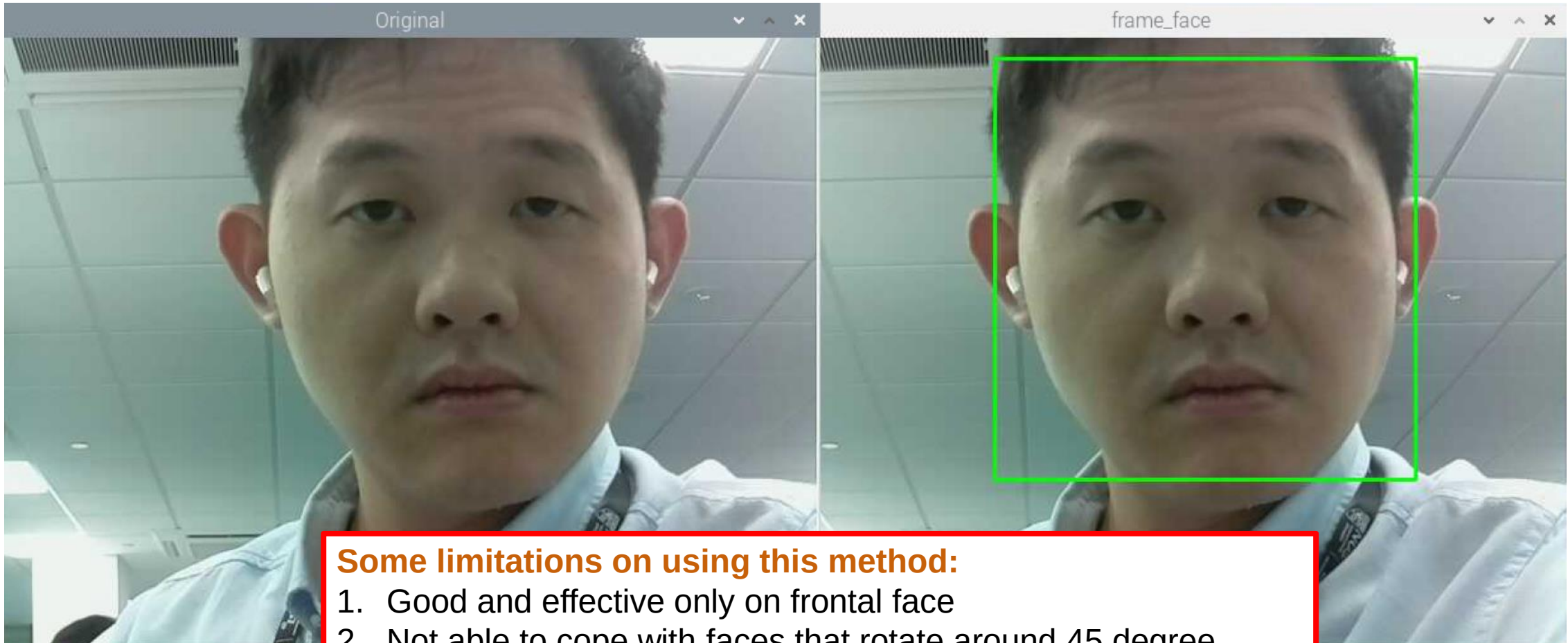
```
# Draw a reactangle box for detected face
frame_face = frame.copy()
for (x,y,w,h) in face_dect:
    cv2.rectangle(frame_face, (x,y), (x+w,y+h), (0,255,0), 2)

# Show the detected face
cv2.imshow('frame_face',frame_face)

# Press 'q' to quit the program
if cv2.waitKey(1) & 0xff == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

1.6. Region of Interest (ROI) Detection

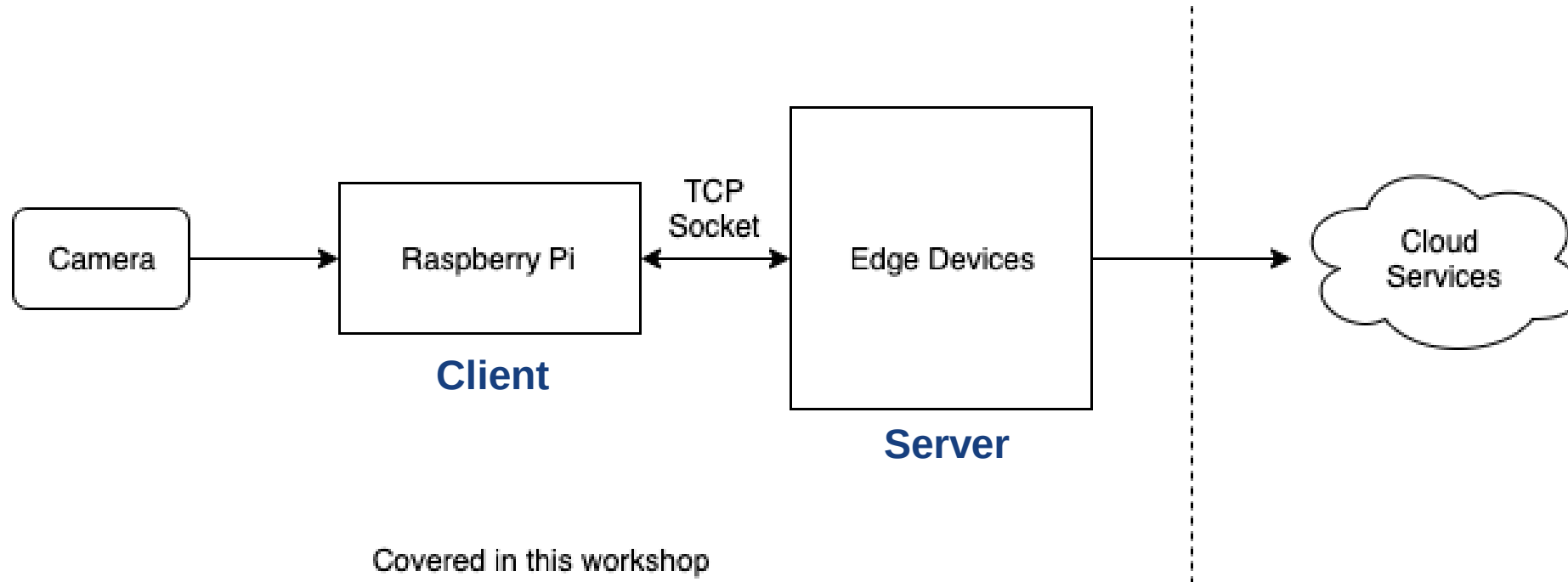


Some limitations on using this method:

1. Good and effective only on frontal face
2. Not able to cope with faces that rotate around 45 degree
3. Sensitive to lighting conditions

1.7. Communication with Edge Server

In this part, we will be **sending image(s) from Raspberry Pi to edge devices via TCP socket** as shown below. The image is resized before sending out to minimize the bandwidth usage.



1.7. Communication with Edge Server

Following is the **client code** to be run in Raspberry Pi (with the camera). You can treat it as the device (such as CCTV) to **capture the image from camera, resizing the image and send it out to edge device**:

```
import socket
import cv2
import pickle
import struct

# Replace this with your server IP address
ip = '192.168.XX.XX'
port = 12345

s = socket.socket(socket.AF_INET , socket.SOCK_STREAM)
s.connect((ip, port))
print('Connected to server')
connection = s.makefile('wb')

try:
    # To capture the image
    cap = cv2.VideoCapture(0)
    img_counter = 0

    # Set JPEG Quality; Range from 0 to 100 (100 best)
    encode_param = [int(cv2.IMWRITE_JPEG_QUALITY),90]

    While True:

        ret, frame = cap.read()

        # Read the image from memory
        # imencode compresses the image and stores it in the memory buffer
        result, frame = cv2.imencode('.jpg', frame, encode_param)

        # Send the image to edge device
        data = pickle.dumps(frame, 0)
        size = len(data)
        s.sendall(struct.pack(">L", size) + data)
        img_counter += 1

    finally:
        print('Sent')
        s.close()
```

1.7. Communication with Edge Server

Following is the server code. You can run this server code on another Raspberry Pi or your computer/laptop. You can treat it as the edge node (such as gateway) to receive the image from Raspberry Pi and save it:

```
import cv2
import socket
import numpy as np
import pickle
import struct

# Replace this with your host IP address
host = '0.0.0.0'
port = 12345

s = socket.socket(socket.AF_INET , socket.SOCK_STREAM)
s.bind((host, port))
s.listen(5)

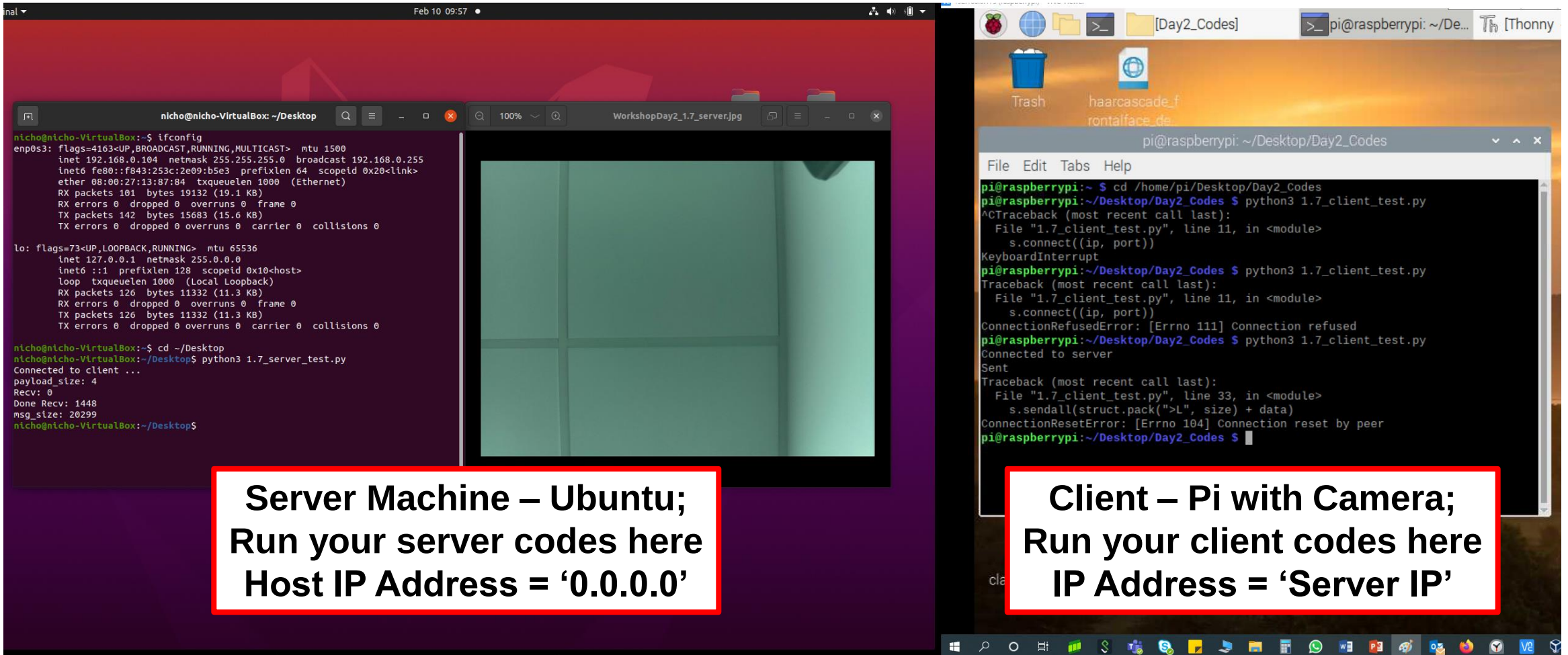
conn,addr = s.accept()
print("Connected to client...")

try:
    name = str('image')+'.jpg'
    data = b""

    payload_size = struct.calcsize(">L")
    print("payload_size: {}".format(payload_size))
    while len(data) < payload_size:
        print("Recv: {}".format(len(data)))
        data += conn.recv(4096)
    print("Done Recv: {}".format(len(data)))
    packed_msg_size = data[:payload_size]
    data = data[payload_size:]
    msg_size = struct.unpack(">L", packed_msg_size)[0]
    print("msg_size: {}".format(msg_size))
    while len(data) < msg_size:
        data += conn.recv(4096)
    frame_data = data[:msg_size]
    data = data[msg_size:]

    frame=pickle.loads(frame_data, fix_imports=True, encoding="bytes")
    frame = cv2.imdecode(frame, cv2.IMREAD_COLOR)
    cv2.imwrite(name,frame)
    cv2.waitKey(1)
```

1.7. Communication with Edge Server



The image shows two side-by-side screenshots. The left screenshot is of a terminal window on a server machine (Ubuntu) with the title 'nicho@nicho-VirtualBox: ~/Desktop'. It displays the output of the 'ifconfig' command for the 'enp0s3' and 'lo' interfaces. The 'enp0s3' interface has an IP address of 192.168.0.104. Below the terminal output is a red-bordered box with the text: 'Server Machine – Ubuntu; Run your server codes here Host IP Address = ‘0.0.0.0’'. The right screenshot is of a Raspberry Pi desktop environment. It shows a terminal window with the title 'pi@raspberrypi: ~/Desktop/Day2_Codes'. The terminal displays the execution of 'python3 1.7_client_test.py', which results in a 'ConnectionRefusedError: [Errno 111] Connection refused' and then a 'ConnectionResetError: [Errno 104] Connection reset by peer'. Below the terminal output is a red-bordered box with the text: 'Client – Pi with Camera; Run your client codes here IP Address = ‘Server IP’'.

Server Machine – Ubuntu;
Run your server codes here
Host IP Address = ‘0.0.0.0’

Client – Pi with Camera;
Run your client codes here
IP Address = ‘Server IP’

Use **\$ sudo apt install python3-opencv** to install opencv
in your server machine if you do not have

Project 1: Improving a Smart Store System

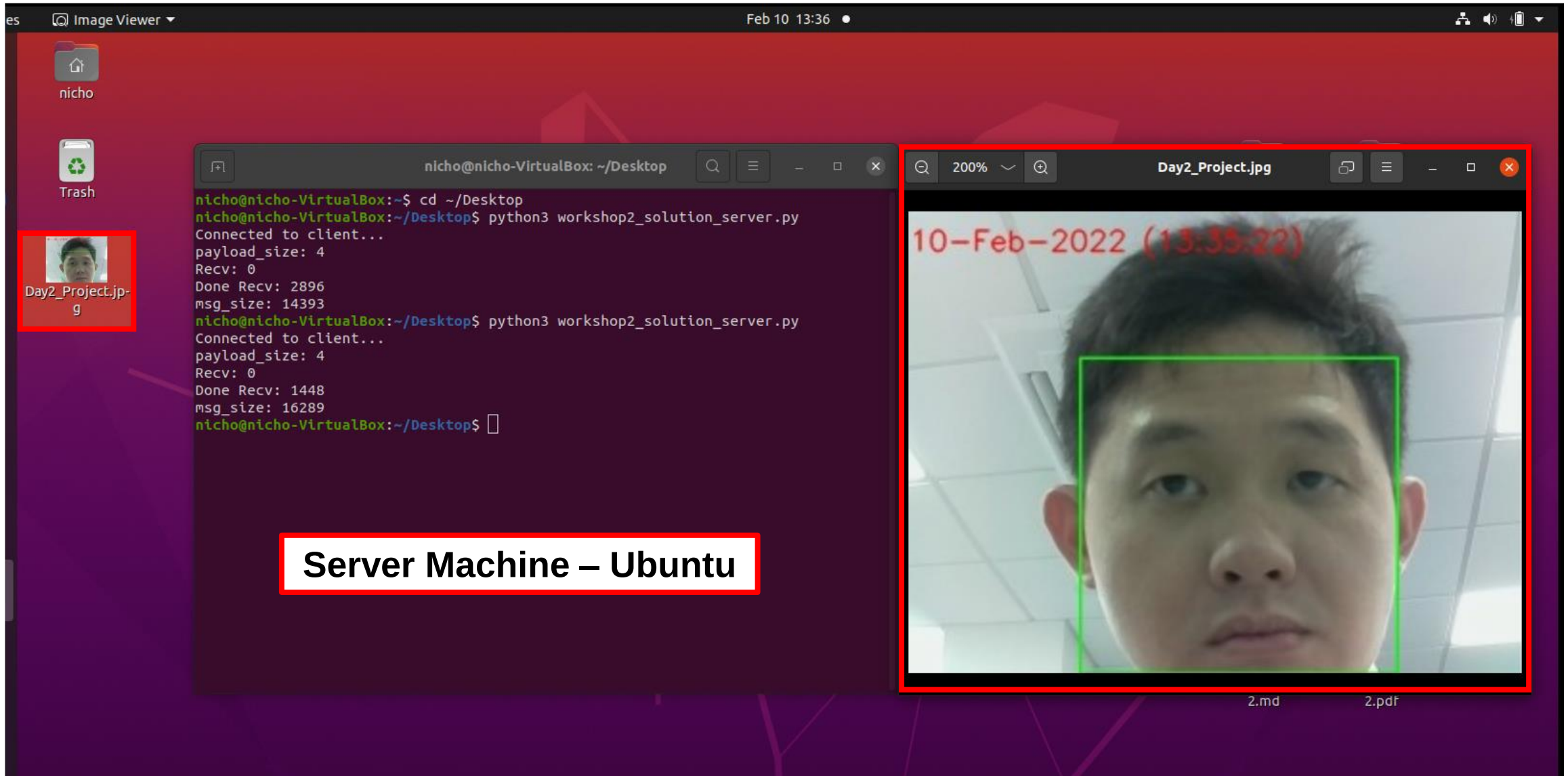
After building the solution for autonomous temperature and humidity detection and alert triggering in Day 1 workshop, Musk's boss was very impressed and he tasked Musk again. **He requested Musk to capture the image of those people entering the store. The photo should label with timestamp and highlighting the face. The images are then required to store in edge node (server) database for recording and further analysis.** Based on what you have learnt so far, can you assist Musk on building the solution?

You may use following code to put text on the image:

```
# Code for putting text (image, text to insert, position, font type,  
font scale, colour, thickness, anti-aliased line)  
cv2.putText(img, 'Minions', (150,410), cv2.FONT_HERSHEY_SIMPLEX, 2.5,  
(0,0,255), 5, cv2.LINE_AA)
```

You are required to submit the python code(s) to Luminus.

Project 1: Improving a Smart Store System



Server Machine – Ubuntu

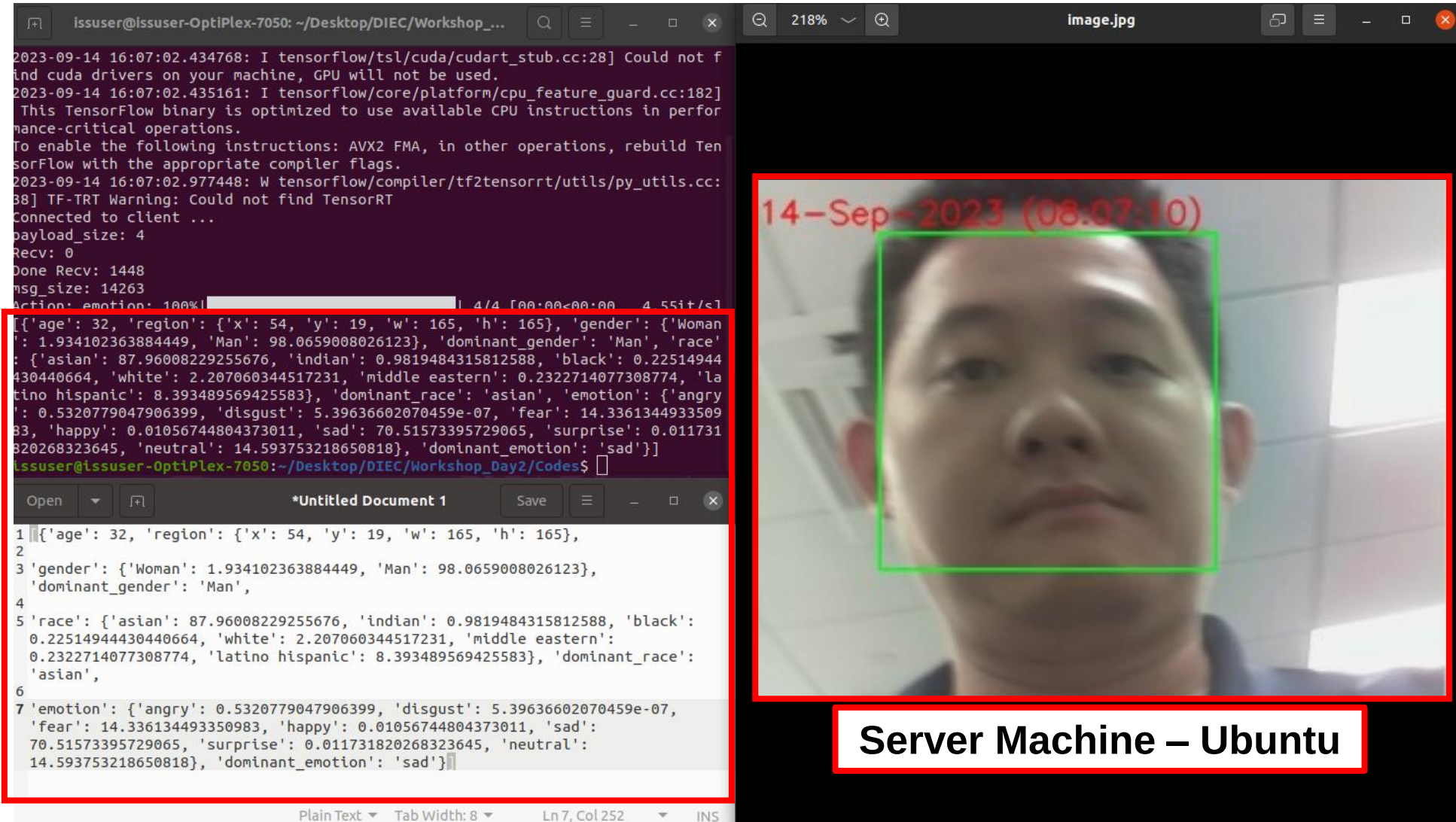
Project 2: Harnessing the computing power of the edge node

Continuing from Project 1, in order to impress his boss, Musk wants to **utilize the captured images in the edge node (server) to do further processing; you can use the same client codes done in Project 1**. You are to help him **develop such a system of your choice**; you can refer to github sources online to do so. **Some examples of further processing you can consider; to estimate**

- Age,
- Gender,
- Race and/or
- Emotions

You are required to submit your results in Canvas

Project 2: Harnessing the computing power of the edge node



The screenshot shows a terminal window on the left and a video feed on the right. The terminal window displays the output of a TensorFlow application, including a warning about GPU drivers and a JSON object representing the results of a face analysis. The video feed shows a man's face with a green bounding box and a red timestamp "14-Sep-2023 (08:07:10)".

Terminal Output:

```
2023-09-14 16:07:02.434768: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2023-09-14 16:07:02.435161: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2023-09-14 16:07:02.977448: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
Connected to client ...
payload_size: 4
Recv: 0
Done Recv: 1448
msg_size: 14263
Action: emotion: 100%
[[{'age': 32, 'region': {'x': 54, 'y': 19, 'w': 165, 'h': 165}, 'gender': {'Woman': 1.934102363884449, 'Man': 98.0659008026123}, 'dominant_gender': 'Man', 'race': {'asian': 87.96008229255676, 'indian': 0.9819484315812588, 'black': 0.22514944430440664, 'white': 2.207060344517231, 'middle eastern': 0.2322714077308774, 'latino hispanic': 8.393489569425583}, 'dominant_race': 'asian', 'emotion': {'angry': 0.5320779047906399, 'disgust': 5.39636602070459e-07, 'fear': 14.336134493350983, 'happy': 0.01056744804373011, 'sad': 70.51573395729065, 'surprise': 0.011731820268323645, 'neutral': 14.593753218650818}, 'dominant_emotion': 'sad'}]]
issuser@issuser-OptiPlex-7050: ~/Desktop/DIEC/Workshop_Day2/Codes$
```

Video Feed:

14-Sep-2023 (08:07:10)

Server Machine – Ubuntu